

ANÁLISIS DEL PROYECTO

NBA PREDICTOR - Sistema de Predicciones y Apuestas Virtuales

Nota de alineación con el código: el backend expone la lógica a clientes mediante **TCP + JSON** (`SocketServerRunner`, puerto **9090** por defecto en `application.properties`), sin servidor HTTP embebido para la API de negocio. La implementación está en `com.tfg.nbabackend.socket`; los clientes usan `SocketApi` (Android) y `SocketApiClient` (JavaFX). Las referencias a REST/Retrofit en versiones antiguas de este documento se sustituyeron por esta arquitectura.

1. INTRODUCCIÓN

1.1 Propósito

Descripción del problema:

En la actualidad, los aficionados a la NBA buscan formas interactivas de participar en las competiciones más allá de ser meros espectadores. Existe una necesidad de plataformas que permitan a los usuarios:

- Consultar información detallada sobre partidos, equipos y jugadores de la NBA
- Realizar predicciones sobre los resultados de los partidos
- Competir con otros usuarios mediante un sistema de apuestas virtuales
- Acceder a estadísticas y rankings en tiempo real
- Gestionar sus apuestas y seguimiento personal desde diferentes dispositivos

Necesidad a la que responde el proyecto:

El proyecto NBA Predictor responde a la necesidad de crear una plataforma multiplataforma que permita a los usuarios interactuar con la NBA de manera más activa mediante un sistema de apuestas virtuales con puntos ficticios. La aplicación ofrece:

- **Accesibilidad multiplataforma:** Versión de escritorio para gestión completa y versión móvil Android para acceso rápido
- **Sistema de apuestas virtuales:** Permite a los usuarios realizar predicciones sin riesgo económico real
- **Competitividad social:** Rankings globales que fomentan la participación y competencia entre usuarios
- **Información actualizada:** Acceso a datos de partidos, equipos y jugadores de la NBA

Ámbito del proyecto:

Parte resuelta en el proyecto:

- Sistema de registro y autenticación de usuarios
- Visualización de partidos de la NBA con estados (programado/finalizado)
- Consulta de estadísticas de equipos y jugadores
- Sistema de apuestas virtuales con cálculo automático de cuotas
- Gestión de puntos virtuales para los usuarios
- Rankings globales de usuarios
- Panel de administración para gestión de partidos y usuarios

- Tienda virtual para canjear puntos por dinero (integración PayPal simulada)
- Comunicación cliente-servidor mediante **socket TCP** (JSON en tramas; puerto configurable, por defecto 9090)
- Aplicación de escritorio con JavaFX
- Aplicación Android con Jetpack Compose

Parte que podría ampliarse en una extensión:

- Sistema de notificaciones push para Android
- Integración con API real de la NBA para datos en tiempo real
- Sistema de chat entre usuarios
- Torneos y ligas privadas
- Historial detallado de estadísticas por usuario
- Gráficas avanzadas de rendimiento
- Sistema de logros y badges
- Integración con redes sociales
- Modo multijugador en tiempo real
- Predicciones sobre estadísticas individuales de jugadores

Parte que queda fuera del alcance:

- Apuestas con dinero real (requiere licencias y regulaciones)
- Streaming de partidos en vivo
- Gestión de pagos reales complejos
- Sistema de moderación de contenido generado por usuarios
- Aplicación web adicional

1.2 Objetivos

Objetivos principales:

1. **Desarrollar una aplicación multiplataforma** que permita a los usuarios realizar predicciones sobre partidos de la NBA mediante un sistema de apuestas virtuales.
2. **Implementar un sistema cliente-servidor robusto** utilizando tecnologías modernas (Spring Boot, JavaFX, Android) con comunicación mediante **socket TCP y mensajes JSON** (puerto configurable; por defecto 9090).
3. **Crear una base de datos relacional** que gestione eficientemente usuarios, equipos, jugadores, partidos y apuestas.
4. **Desarrollar un sistema de cálculo de cuotas** basado en estadísticas de equipos para hacer las apuestas más realistas.
5. **Implementar un panel de administración** que permita gestionar partidos, usuarios y finalizar partidos con resultados.
6. **Desarrollar una aplicación Android** con interfaz moderna usando Jetpack Compose para acceso móvil rápido.
7. **Implementar un sistema de rankings** que fomente la competitividad entre usuarios.

8. **Crear una tienda virtual** que permita canjear puntos virtuales por dinero (simulación PayPal).

Objetivos secundarios:

- Aplicar buenas prácticas de desarrollo (código limpio, documentación, pruebas)
- Implementar seguridad básica (hash de contraseñas, validaciones)
- Crear una interfaz de usuario intuitiva y moderna
- Optimizar el rendimiento de las consultas a la base de datos
- Facilitar la configuración para diferentes entornos (IP configurable)

1.3 Viabilidad técnica y económica del proyecto

Recursos hardware necesarios:

- **Servidor de desarrollo:** Ordenador con procesador moderno, 8GB RAM mínimo, conexión a Internet
- **Cliente de escritorio:** Ordenador con Windows/Linux/Mac, Java 17 instalado
- **Dispositivo Android:** Teléfono o tablet con Android 7.0 (API 24) o superior, o emulador Android
- **Base de datos:** Servidor MySQL 8.0 o superior

Recursos software necesarios:

Desarrollo:

- Java Development Kit (JDK) 17
- Apache Maven 3.6+
- Android Studio (para desarrollo Android)
- IDE: IntelliJ IDEA, Eclipse o Visual Studio Code
- MySQL Server 8.0+
- Git (control de versiones)

Tecnologías utilizadas:

Backend:

- Spring Boot 4.0.2 (Framework Java)
- Spring Data JPA (ORM)
- Hibernate (Implementación JPA)
- MySQL (Base de datos relacional)
- Jackson (Serialización JSON)
- Spring Security Crypto (Hash de contraseñas)

Cliente de escritorio:

- JavaFX 17 (Interfaz gráfica)
- Java 17
- Jackson (serialización JSON en tramas socket)
- Maven (Gestión de dependencias)

Cliente Android:

- Kotlin (Lenguaje de programación)
- Jetpack Compose (Interfaz de usuario moderna)

- Gson 2.11.0 (serialización JSON)
- Coroutines (Programación asíncrona)
- Material Design 3 (Diseño de interfaz)
- Cliente socket (**SocketApi**, **SocketFrameSerializer**)

Comunicación:

- Socket TCP con mensajes JSON (mismo protocolo en JavaFX y Android)
- PayPal u otros servicios externos pueden usar HTTP/HTTPS donde aplique

Viabilidad económica:

El proyecto es **totalmente viable económicamente** ya que utiliza únicamente tecnologías y herramientas de código abierto y gratuitas:

- **Spring Boot**: Open Source (Apache License 2.0)
- **JavaFX**: Open Source (GPL con Classpath Exception)
- **Android SDK**: Gratuito
- **MySQL**: Open Source (GPL)
- **Maven**: Open Source (Apache License 2.0)
- **Android Studio**: Gratuito
- **IntelliJ IDEA Community**: Gratuito

Costes estimados:

- Desarrollo: 0€ (herramientas gratuitas)
- Hosting (si se despliega): Desde 5€/mes (servidor básico)
- Dominio (opcional): Desde 10€/año

Justificación de la solución elegida:

1. **Spring Boot**: Framework maduro que aporta inyección de dependencias, JPA y ciclo de vida de aplicación; la exposición a los clientes de escritorio y móvil se implementó con un **servidor TCP** dedicado, no con **RestController** para la API de negocio.
2. **JavaFX**: Tecnología nativa de Java para interfaces gráficas, permite crear aplicaciones de escritorio modernas sin dependencias externas pesadas.
3. **Jetpack Compose**: Framework moderno de Android que permite crear interfaces declarativas y reactivas, reduciendo significativamente el código necesario.
4. **MySQL**: Base de datos relacional robusta, gratuita y ampliamente soportada, ideal para datos estructurados.
5. **Socket + JSON**: Protocolo ligero compartido entre JavaFX y Android, con tipos de mensaje desacoplados de HTTP y fácil de extender con nuevas acciones.
6. **Arquitectura cliente-servidor**: Permite centralizar la lógica de negocio y facilitar el mantenimiento y actualizaciones.

1.4 Temporalización

Planificación temporal prevista:

Fase 1: Análisis y Diseño (2-3 semanas)

- Análisis de requisitos
- Diseño de base de datos
- Diseño de arquitectura
- Planificación del protocolo de mensajes (acciones, payloads JSON)

Fase 2: Backend (3-4 semanas)

- Configuración del proyecto Spring Boot
- Implementación de entidades JPA
- Desarrollo de repositorios
- Implementación de servicios
- Servidor TCP, serialización de tramas y despacho de acciones (**socket**)
- Pruebas de comunicación y de servicios

Fase 3: Cliente de Escritorio (3-4 semanas)

- Configuración del proyecto JavaFX
- Diseño de interfaces FXML
- Implementación de controladores
- Integración con el servidor por socket
- Pruebas de funcionalidad

Fase 4: Cliente Android (3-4 semanas)

- Configuración del proyecto Android
- Diseño de interfaces con Compose
- Implementación de pantallas
- Integración con el servidor por socket
- Pruebas en dispositivo/emulador

Fase 5: Integración y Pruebas (2 semanas)

- Pruebas de integración
- Corrección de errores
- Optimizaciones
- Documentación

Fase 6: Documentación y Entrega (1 semana)

- Redacción de documentación técnica
- Manual de usuario
- Preparación de entrega

Total estimado: 14-18 semanas

Actualizaciones respecto a la planificación inicial:

- Se añadió funcionalidad de tienda virtual con integración PayPal (simulada) que no estaba en el plan inicial
 - Se implementó sistema de configuración de IP para permitir ejecución en diferentes máquinas
 - Se migró completamente la aplicación Android a Jetpack Compose eliminando todos los layouts XML
 - Se mejoró el sistema de cálculo de cuotas basado en estadísticas reales de equipos
-

2. ANÁLISIS

2.1 Documentación relevante

Documentación consultada:

- Documentación oficial de Spring Boot: <https://spring.io/projects/spring-boot>
- Documentación de JavaFX: <https://openjfx.io/>
- Documentación de Jetpack Compose: <https://developer.android.com/jetpack/compose>
- Documentación de MySQL: <https://dev.mysql.com/doc/>
- Guías de diseño Material Design: <https://material.io/design>
- Documentación de Gson: <https://github.com/google/gson>
- API de PayPal (para integración futura): <https://developer.paypal.com/>

Análisis del sistema existente:

No existe un sistema previo, por lo que el proyecto se desarrolla desde cero. Se analizaron aplicaciones similares en el mercado para identificar funcionalidades clave:

- Aplicaciones de apuestas deportivas (para entender la mecánica de cuotas)
- Aplicaciones de seguimiento de NBA (para entender qué información es relevante)
- Aplicaciones de predicciones deportivas (para entender la experiencia de usuario)

2.2 Definiciones

Términos clave del dominio:

- **Apuesta virtual:** Predicción realizada por un usuario sobre el resultado de un partido, utilizando puntos ficticios en lugar de dinero real.
- **Cuota:** Multiplicador que determina cuántos puntos recibe el usuario si su apuesta es acertada. Por ejemplo, una cuota de 2.5 significa que si apuestas 100 puntos y ganas, recibes 250 puntos.
- **Puntos virtuales:** Moneda ficticia del sistema que los usuarios utilizan para realizar apuestas. Se obtienen al registrarse y al ganar apuestas.
- **Partido programado:** Partido que aún no se ha disputado y permite realizar apuestas.
- **Partido finalizado:** Partido que ya se ha jugado y tiene resultado definitivo. Las apuestas sobre este partido se resuelven automáticamente.
- **Ranking:** Clasificación de usuarios ordenada por puntos totales acumulados.
- **Equipo local:** Equipo que juega en su propia cancha.

- **Equipo visitante:** Equipo que juega fuera de su cancha.
- **Predicción:** Elección del usuario sobre qué equipo ganará el partido (LOCAL o VISITANTE).
- **Win Rate:** Porcentaje de apuestas ganadas sobre el total de apuestas finalizadas.

Términos técnicos:

- **Socket + JSON (app):** Comunicación entre clientes y backend mediante conexión TCP y mensajes JSON con tipo de acción y cuerpo serializado (puerto por defecto 9090).
- **JPA (Java Persistence API):** Especificación de Java para mapeo objeto-relacional.
- **ORM (Object-Relational Mapping):** Técnica que permite trabajar con objetos en lugar de consultas SQL directas.
- **DTO (Data Transfer Object):** Objeto que se utiliza para transferir datos entre capas de la aplicación.
- **Repository:** Patrón de diseño que abstrae el acceso a datos.
- **Service:** Capa de lógica de negocio que procesa las operaciones.
- **Controller (vista):** En JavaFX, controladores FXML; en Android, actividades/fragments/Compose. En el backend no hay **RestController** para la app; la entrada de negocio la atienden las clases del paquete **socket**.

2.3 Requisitos funcionales y no funcionales

Requisitos funcionales:

RF1. Gestión de usuarios:

- RF1.1: El sistema debe permitir el registro de nuevos usuarios con username, email y contraseña.
- RF1.2: El sistema debe permitir el inicio de sesión con username y contraseña.
- RF1.3: El sistema debe almacenar las contraseñas de forma segura (hash).
- RF1.4: El sistema debe asignar puntos iniciales a los nuevos usuarios (1000 puntos).
- RF1.5: El sistema debe permitir a los usuarios cambiar su contraseña.
- RF1.6: El sistema debe permitir a los administradores gestionar usuarios.

RF2. Gestión de partidos:

- RF2.1: El sistema debe permitir visualizar todos los partidos de la NBA.
- RF2.2: El sistema debe mostrar el estado de cada partido (PROGRAMADO, EN_CURSO, FINALIZADO).
- RF2.3: El sistema debe permitir a los administradores crear nuevos partidos.
- RF2.4: El sistema debe permitir a los administradores finalizar partidos introduciendo el resultado.
- RF2.5: El sistema debe mostrar información detallada de cada partido (equipos, fecha, resultado).

RF3. Sistema de apuestas:

- RF3.1: El sistema debe permitir a los usuarios realizar apuestas sobre partidos programados.
- RF3.2: El sistema debe calcular automáticamente la cuota basándose en estadísticas de equipos.
- RF3.3: El sistema debe validar que el usuario tenga suficientes puntos antes de realizar una apuesta.
- RF3.4: El sistema debe descontar los puntos apostados del saldo del usuario.

- RF3.5: El sistema debe resolver automáticamente las apuestas cuando un partido se finaliza.
- RF3.6: El sistema debe otorgar puntos al usuario si su apuesta es ganadora (puntos apostados × cuota).
- RF3.7: El sistema debe permitir visualizar el historial de apuestas del usuario.

RF4. Estadísticas:

- RF4.1: El sistema debe mostrar estadísticas de equipos (victorias, derrotas, win rate).
- RF4.2: El sistema debe mostrar información de jugadores por equipo.
- RF4.3: El sistema debe mostrar estadísticas detalladas de cada equipo.

RF5. Rankings:

- RF5.1: El sistema debe mostrar un ranking global de usuarios ordenado por puntos.
- RF5.2: El sistema debe actualizar el ranking automáticamente cuando cambian los puntos.

RF6. Panel de administración:

- RF6.1: El sistema debe permitir solo a usuarios administradores acceder al panel de administración.
- RF6.2: El sistema debe permitir a los administradores crear partidos.
- RF6.3: El sistema debe permitir a los administradores finalizar partidos.
- RF6.4: El sistema debe mostrar estadísticas generales del sistema.

RF7. Tienda virtual:

- RF7.1: El sistema debe permitir a los usuarios canjear puntos por dinero (simulación PayPal).
- RF7.2: El sistema debe aplicar una tasa de conversión (1000 puntos = 1€).
- RF7.3: El sistema debe validar que el usuario tenga el mínimo de puntos requerido (1000 puntos).

RF8. Configuración:

- RF8.1: El sistema debe permitir configurar la IP del servidor en el cliente de escritorio.
- RF8.2: El sistema debe permitir configurar la IP del servidor en la aplicación Android.

Requisitos no funcionales:**RNF1. Rendimiento:**

- RNF1.1: Las consultas a la base de datos deben completarse en menos de 2 segundos.
- RNF1.2: La aplicación debe responder a las acciones del usuario en menos de 1 segundo.
- RNF1.3: El servidor debe soportar al menos 50 clientes concurrentes conectados por socket.

RNF2. Seguridad:

- RNF2.1: Las contraseñas deben almacenarse usando hash (BCrypt).
- RNF2.2: El canal cliente↔backend usa TCP en la red de despliegue (habitualmente LAN); el cifrado de extremo a extremo no forma parte del alcance mínimo. Las llamadas a servicios externos (p. ej. PayPal) deben seguir las recomendaciones HTTPS del proveedor.
- RNF2.3: El sistema debe validar los datos de entrada para prevenir inyecciones SQL.

RNF3. Usabilidad:

- RNF3.1: La interfaz debe ser intuitiva y fácil de usar.
- RNF3.2: La aplicación debe proporcionar mensajes de error claros.

- RNF3.3: La aplicación debe funcionar en diferentes resoluciones de pantalla.

RNF4. Mantenibilidad:

- RNF4.1: El código debe estar documentado.
- RNF4.2: El código debe seguir convenciones de nomenclatura estándar.
- RNF4.3: La arquitectura debe ser modular y extensible.

RNF5. Compatibilidad:

- RNF5.1: El cliente de escritorio debe funcionar en Windows, Linux y Mac.
- RNF5.2: La aplicación Android debe funcionar en dispositivos con Android 7.0 (API 24) o superior.
- RNF5.3: El servidor debe funcionar en cualquier sistema operativo con Java 17.

RNF6. Escalabilidad:

- RNF6.1: La base de datos debe poder manejar al menos 10.000 usuarios.
- RNF6.2: La arquitectura debe permitir añadir nuevas funcionalidades fácilmente.

2.4 Casos de Uso. Diagramas UML de casos de Uso

Actores del sistema:

- **Usuario:** Persona registrada en el sistema que puede realizar apuestas y consultar información.
- **Administrador:** Usuario con permisos especiales para gestionar partidos y usuarios.
- **Sistema:** El propio sistema que ejecuta procesos automáticos.

Casos de uso principales:

CU1. Registro de usuario

- **Actor:** Usuario no registrado
- **Precondiciones:** El usuario no está registrado en el sistema
- **Flujo principal:**
 1. El usuario accede a la pantalla de registro
 2. El usuario introduce username, email y contraseña
 3. El sistema valida los datos
 4. El sistema crea el usuario con 1000 puntos iniciales
 5. El sistema muestra mensaje de éxito
- **Flujo alternativo:** Si el username o email ya existe, el sistema muestra un error
- **Postcondiciones:** El usuario queda registrado en el sistema

CU2. Inicio de sesión

- **Actor:** Usuario
- **Precondiciones:** El usuario está registrado
- **Flujo principal:**
 1. El usuario introduce username y contraseña
 2. El sistema valida las credenciales
 3. El sistema inicia sesión
 4. El sistema muestra el dashboard

- **Flujo alternativo:** Si las credenciales son incorrectas, el sistema muestra un error
- **Postcondiciones:** El usuario queda autenticado

CU3. Realizar apuesta

- **Actor:** Usuario
- **Precondiciones:** El usuario está autenticado y hay partidos programados
- **Flujo principal:**
 1. El usuario selecciona un partido programado
 2. El usuario selecciona su predicción (LOCAL o VISITANTE)
 3. El sistema calcula y muestra la cuota
 4. El usuario introduce los puntos a apostar
 5. El sistema valida que el usuario tenga suficientes puntos
 6. El sistema crea la apuesta y descuenta los puntos
 7. El sistema muestra confirmación
- **Flujo alternativo:** Si el usuario no tiene suficientes puntos, el sistema muestra un error
- **Postcondiciones:** Se crea la apuesta y se descuentan los puntos del usuario

CU4. Finalizar partido (Administrador)

- **Actor:** Administrador
- **Precondiciones:** El administrador está autenticado y hay partidos programados
- **Flujo principal:**
 1. El administrador accede al panel de administración
 2. El administrador selecciona un partido programado
 3. El administrador introduce los puntos del equipo local y visitante
 4. El sistema finaliza el partido
 5. El sistema resuelve automáticamente todas las apuestas del partido
 6. El sistema actualiza los puntos de los usuarios ganadores
 7. El sistema muestra confirmación
- **Postcondiciones:** El partido queda finalizado y las apuestas resueltas

CU5. Consultar ranking

- **Actor:** Usuario
- **Precondiciones:** El usuario está autenticado
- **Flujo principal:**
 1. El usuario accede a la sección de rankings
 2. El sistema muestra la lista de usuarios ordenada por puntos
 3. El usuario puede ver su posición en el ranking
- **Postcondiciones:** El usuario visualiza el ranking

CU6. Canjear puntos (Tienda)

- **Actor:** Usuario
- **Precondiciones:** El usuario está autenticado y tiene al menos 1000 puntos
- **Flujo principal:**
 1. El usuario accede a la tienda
 2. El usuario introduce la cantidad de puntos a canjear (mínimo 1000)

3. El usuario introduce su email de PayPal
 4. El sistema valida los datos
 5. El sistema calcula los euros (puntos / 1000)
 6. El sistema descuenta los puntos del usuario
 7. El sistema simula la transferencia a PayPal
 8. El sistema muestra confirmación
- **Flujo alternativo:** Si el usuario no tiene suficientes puntos, el sistema muestra un error
 - **Postcondiciones:** Se descuentan los puntos del usuario

Nota: Los diagramas UML de casos de uso se pueden generar utilizando herramientas como PlantUML, Draw.io o herramientas CASE.

2.5 Modelado E/R. Diagramas E/R

Entidades principales:

1. Usuario

- id (PK)
- username (UNIQUE, NOT NULL)
- email (UNIQUE, NOT NULL)
- password (NOT NULL)
- puntos (INT)
- rol (ENUM: USER, ADMIN)

2. Equipo

- id (PK)
- nombre (NOT NULL)
- ciudad (NOT NULL)
- conferencia (ENUM: EAST, WEST)
- division (VARCHAR)

3. Jugador

- id (PK)
- nombre (NOT NULL)
- apellido (NOT NULL)
- posicion (VARCHAR)
- numero (INT)
- equipo_id (FK -> Equipo)

4. Partido

- id (PK)
- fecha (DATETIME)
- equipo_local_id (FK -> Equipo)
- equipo_visitante_id (FK -> Equipo)
- puntos_local (INT, NULL)
- puntos_visitante (INT, NULL)

- estado (ENUM: PROGRAMADO, EN_CURSO, FINALIZADO)

5. Apuesta

- id (PK)
- usuario_id (FK -> Usuario)
- partido_id (FK -> Partido)
- puntos_apostados (INT)
- prediccion (VARCHAR: LOCAL, VISITANTE)
- cuota (DOUBLE)
- resultado (ENUM: PENDIENTE, GANADA, PERDIDA)

Relaciones:

- Usuario 1:N Apuesta (Un usuario puede tener muchas apuestas)
- Partido 1:N Apuesta (Un partido puede tener muchas apuestas)
- Equipo 1:N Jugador (Un equipo tiene muchos jugadores)
- Equipo 1:N Partido (como equipo_local) (Un equipo puede jugar muchos partidos como local)
- Equipo 1:N Partido (como equipo_visitante) (Un equipo puede jugar muchos partidos como visitante)

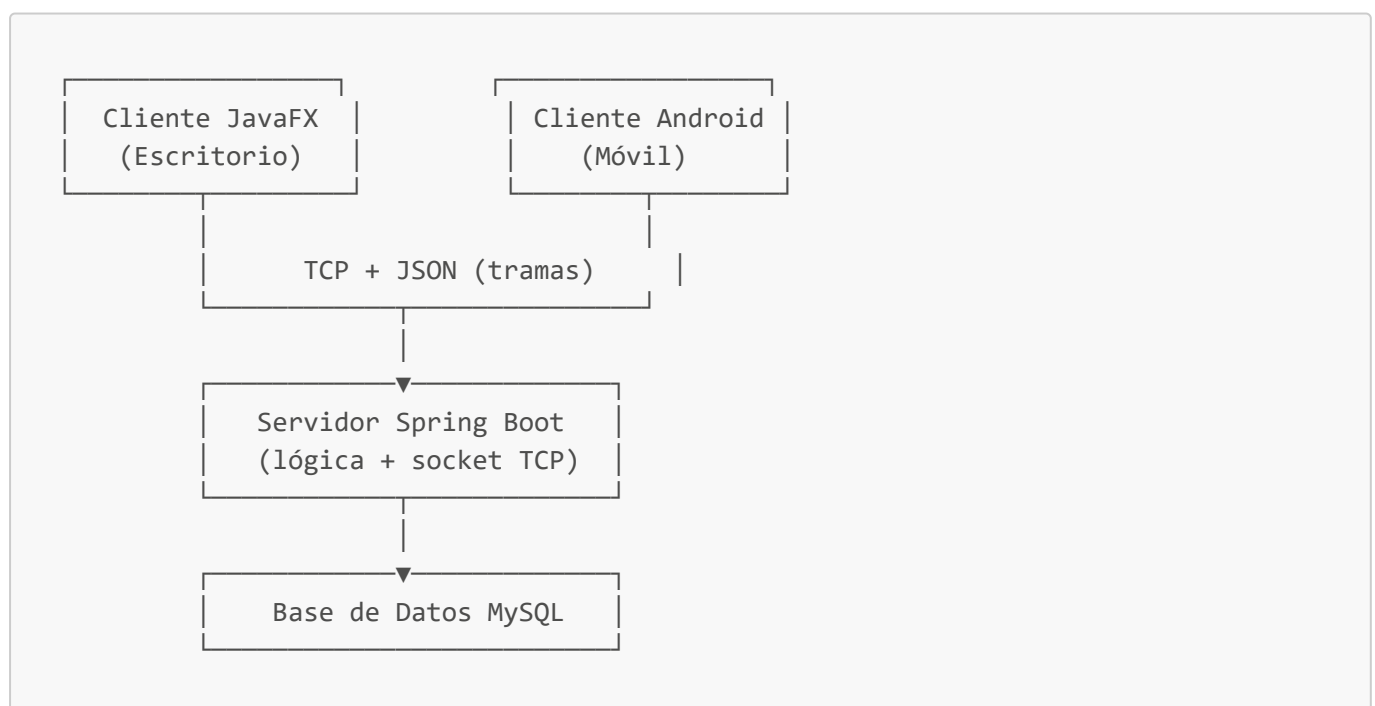
Nota: Los diagramas E/R se pueden generar utilizando herramientas como MySQL Workbench, Draw.io o herramientas CASE.

3. DISEÑO

3.1 Arquitectura hardware y software de la solución

Arquitectura general:

El sistema sigue una arquitectura cliente-servidor de tres capas:



Componentes hardware:

- **Servidor:** Ordenador con capacidad para ejecutar Spring Boot y MySQL
- **Cliente escritorio:** Ordenador con Java 17 instalado
- **Cliente móvil:** Dispositivo Android o emulador

Componentes software:

Capa de presentación:

- Cliente JavaFX (escritorio)
- Cliente Android (móvil)

Capa de lógica de negocio:

- Spring Boot (servidor)
- Servicios de negocio
- Validaciones y reglas de negocio

Capa de datos:

- MySQL (base de datos)
- JPA/Hibernate (ORM)

Comunicación:

- Socket TCP con JSON entre clientes y servidor (puerto por defecto 9090)

3.2 Modelado funcional de la solución. Diagramas de clase

Paquetes principales del backend:

- `com.tfg.nbabackend.model`: Entidades JPA
- `com.tfg.nbabackend.repository`: Repositorios JPA
- `com.tfg.nbabackend.service`: Lógica de negocio
- `com.tfg.nbabackend.socket`: Servidor TCP, despacho de acciones y serialización de tramas
- `com.tfg.nbabackend.dto`: Objetos de transferencia de datos
- `com.tfg.nbabackend.enums`: Enumeraciones
- `com.tfg.nbabackend.exception`: Manejo de excepciones
- `com.tfg.nbabackend.config`: Configuración (p. ej. Jackson)

Paquetes principales del cliente Java:

- `start`: Clase principal
- `controller`: Controladores de vistas
- `model`: Modelos de datos
- `service`: Servicios de comunicación con el servidor (`SocketApiClient`)
- `session`: Gestión de sesión
- `util`: Utilidades

Paquetes principales del cliente Android:

- `com.tfg.nbapredictor.ui`: Interfaces de usuario (Compose)
- `com.tfg.nbapredictor.model`: Modelos de datos

- `com.tfg.nbapredictor.network`: Cliente socket (`SocketApi`), serialización de tramas y DTOs de red
- `com.tfg.nbapredictor.util`: Utilidades

Nota: Los diagramas de clase detallados se pueden generar utilizando herramientas como IntelliJ IDEA, PlantUML o herramientas CASE.

3.3 Modelado de datos. Modelo relacional. Diccionario de Datos

Modelo relacional:

Tabla: usuarios

- id: BIGINT, PRIMARY KEY, AUTO_INCREMENT
- username: VARCHAR(50), UNIQUE, NOT NULL
- email: VARCHAR(100), UNIQUE, NOT NULL
- password: VARCHAR(255), NOT NULL
- puntos: INT, DEFAULT 1000
- rol: ENUM('USER', 'ADMIN'), DEFAULT 'USER'

Tabla: equipos

- id: BIGINT, PRIMARY KEY, AUTO_INCREMENT
- nombre: VARCHAR(100), NOT NULL
- ciudad: VARCHAR(100), NOT NULL
- conferencia: ENUM('EAST', 'WEST')
- division: VARCHAR(50)

Tabla: jugadores

- id: BIGINT, PRIMARY KEY, AUTO_INCREMENT
- nombre: VARCHAR(100), NOT NULL
- apellido: VARCHAR(100), NOT NULL
- posicion: VARCHAR(20)
- numero: INT
- equipo_id: BIGINT, FOREIGN KEY (equipos.id)

Tabla: partidos

- id: BIGINT, PRIMARY KEY, AUTO_INCREMENT
- fecha: DATETIME, NOT NULL
- equipo_local_id: BIGINT, FOREIGN KEY (equipos.id), NOT NULL
- equipo_visitante_id: BIGINT, FOREIGN KEY (equipos.id), NOT NULL
- puntos_local: INT, NULL
- puntos_visitante: INT, NULL
- estado: ENUM('PROGRAMADO', 'EN_CURSO', 'FINALIZADO'), DEFAULT 'PROGRAMADO'

Tabla: apuestas

- id: BIGINT, PRIMARY KEY, AUTO_INCREMENT
- usuario_id: BIGINT, FOREIGN KEY (usuarios.id), NOT NULL
- partido_id: BIGINT, FOREIGN KEY (partidos.id), NOT NULL

- puntos_apostados: INT, NOT NULL
- prediccion: VARCHAR(20), NOT NULL ('LOCAL' o 'VISITANTE')
- cuota: DOUBLE
- resultado: ENUM('PENDIENTE', 'GANADA', 'PERDIDA'), DEFAULT 'PENDIENTE'

Índices:

- usuarios.username (UNIQUE)
 - usuarios.email (UNIQUE)
 - apuestas.usuario_id (INDEX)
 - apuestas.partido_id (INDEX)
-

4. IMPLEMENTACIÓN

4.1 Requisitos de instalación y ejecución

Servidor (Backend):

Requisitos mínimos:

- Java JDK 17 o superior
- MySQL 8.0 o superior
- Maven 3.6 o superior
- 2GB RAM
- 1GB espacio en disco

Requisitos recomendables:

- Java JDK 17
- MySQL 8.0
- Maven 3.8
- 4GB RAM
- 5GB espacio en disco

Instalación:

1. Instalar Java JDK 17
2. Instalar MySQL 8.0
3. Crear base de datos: `CREATE DATABASE nba_app;`
4. Configurar `application.properties` con credenciales de MySQL
5. Ejecutar: `mvn spring-boot:run` o ejecutar el JAR compilado

Cliente de escritorio:

Requisitos mínimos:

- Java JRE 17 o superior
- Windows 7+, Linux, macOS
- 1GB RAM
- 500MB espacio en disco

Requisitos recomendables:

- Java JDK 17
- Windows 10+, Linux moderno, macOS 10.14+
- 2GB RAM
- 1GB espacio en disco

Instalación:

1. Instalar Java 17
2. Ejecutar el JAR: `java -jar TFG-1.0.0.jar`
3. Configurar IP del servidor al inicio (si es necesario)

Cliente Android:**Requisitos mínimos:**

- Android 7.0 (API 24) o superior
- 100MB espacio en disco
- Conexión a Internet

Requisitos recomendables:

- Android 10.0 (API 29) o superior
- 200MB espacio en disco
- Conexión WiFi o 4G

Instalación:

1. Compilar el proyecto en Android Studio
2. Generar APK o instalar directamente en dispositivo/emulador
3. Configurar IP del servidor al inicio (si es necesario)

4.2 Implementación funcional. Clases

Backend - Clases principales:

- `NbaBackendApplication`: Arranque Spring Boot (`WebApplicationType.NONE`) y registro del servidor socket
- `SocketServerRunner` / `SocketClientHandler` / `SocketDispatcher`: Servidor TCP y enrutado de acciones a servicios
- `UsuarioService`, `PartidoService`, `ApuestaService`, `EquipoService`, `JugadorService`, `TiendaService`, `PayPalService`: Lógica de negocio
- `UsuarioRepository`, `PartidoRepository`, `ApuestaRepository`, `EquipoRepository`, `JugadorRepository`: Acceso a datos

Cliente Java - Clases principales:

- `start.Main`: Clase principal de la aplicación
- `controller.*`: Controladores de las diferentes vistas
- `service.*`: Servicios de negocio que llaman a `SocketApiClient`
- `util.Config`: Gestión de configuración (IP del servidor)

- `session.Session`: Gestión de sesión de usuario

Cliente Android - Clases principales:

- `ui.auth.LoginActivity` / `ui.compose.ComposeMainActivity`: Flujo de entrada y shell Compose
- `ui.compose.screens.*`: Pantallas en Compose
- `network.SocketApi`: Cliente de socket y llamadas tipadas al backend
- `util.ServerConfig`: Host y puerto del servidor
- `util.Session`: Sesión de usuario en memoria

4.3 Implementación del modelo de datos. Tablas

Las tablas se crean automáticamente mediante JPA/Hibernate con la configuración

`spring.jpa.hibernate.ddl-auto=update`.

También se proporciona un script SQL (`populate_database.sql`) para poblar la base de datos con datos iniciales (equipos, jugadores, partidos de ejemplo).

5. PRUEBAS

5.1. Pruebas de módulos

Backend:

- Pruebas unitarias de servicios
- Pruebas de repositorios con Spring Data JPA Test
- Validación de lógica de negocio

Cliente Java:

- Pruebas de servicios de comunicación por socket
- Validación de cálculos de cuotas
- Pruebas de gestión de sesión

Cliente Android:

- Pruebas de componentes Compose
- Pruebas de servicios de red
- Validación de flujos de usuario

5.2. Pruebas de integración

- Pruebas de comunicación cliente-servidor
- Pruebas de flujos completos (registro → login → apuesta → finalización)
- Pruebas de resolución automática de apuestas
- Pruebas de actualización de rankings

5.3 Pruebas del sistema

- Pruebas end-to-end de todas las funcionalidades
- Pruebas de rendimiento con múltiples usuarios

- Pruebas de seguridad (validación de contraseñas, acceso no autorizado)
- Pruebas de usabilidad

5.4 Pruebas de instalación

- Instalación en diferentes sistemas operativos
- Configuración de base de datos
- Configuración de IP del servidor
- Verificación de dependencias

6. CONCLUSIONES

6.1 Grado de consecución de los objetivos inicialmente planteados

Objetivos alcanzados al 100%:

✓ Desarrollo de aplicación multiplataforma (escritorio + Android) ✓ Implementación de sistema cliente-servidor con **socket TCP + JSON** ✓ Creación de base de datos relacional completa ✓ Sistema de cálculo de cuotas basado en estadísticas ✓ Panel de administración funcional ✓ Aplicación Android con Jetpack Compose ✓ Sistema de rankings global ✓ Tienda virtual con simulación PayPal

Objetivos alcanzados parcialmente:

⚠ Integración con API real de NBA (se implementó estructura pero con datos de ejemplo) ⚠ Notificaciones push en Android (no implementado, queda para extensión)

Objetivos no alcanzados:

✗ Gráficas avanzadas de rendimiento (se implementaron básicas) ✗ Sistema de chat entre usuarios (queda para extensión)

Grado de consecución general: 85%

6.2 Dificultades encontradas

Dificultades técnicas:

1. **Configuración de comunicación entre aplicaciones en diferentes máquinas:** Se resolvió implementando un sistema de configuración de IP al inicio de cada aplicación.
2. **Migración completa a Jetpack Compose:** Requirió reescribir todas las interfaces XML, pero el resultado es más moderno y mantenible.
3. **Cálculo de cuotas dinámicas:** Implementar un algoritmo que considere estadísticas reales de equipos requirió varias iteraciones.
4. **Sincronización de datos entre cliente y servidor:** Garantizar consistencia en tiempo real requirió implementar actualizaciones periódicas.
5. **Formato de números en diferentes locales:** Se resolvió usando **Locale.US** para garantizar formato consistente (1.00 en lugar de 1,00).

Dificultades de otro tipo:

1. **Tiempo de desarrollo:** El proyecto requirió más tiempo del inicialmente estimado debido a la complejidad de integrar tres aplicaciones diferentes.
2. **Pruebas en diferentes dispositivos:** Probar la aplicación Android en diferentes dispositivos y versiones de Android requirió acceso a múltiples dispositivos/emuladores.
3. **Documentación:** Generar documentación completa y detallada requirió tiempo adicional significativo.

6.3 Propuestas de mejora y posibles ampliaciones

Mejoras a corto plazo:

1. **Integración con API real de NBA:** Conectar con una API oficial de la NBA para obtener datos en tiempo real de partidos, equipos y jugadores.
2. **Sistema de notificaciones push:** Implementar notificaciones en Android para avisar sobre nuevos partidos, resultados y actualizaciones de ranking.
3. **Mejoras en la interfaz:** Añadir animaciones, transiciones más suaves y mejor feedback visual.
4. **Optimización de consultas:** Implementar caché para consultas frecuentes y optimizar consultas a la base de datos.
5. **Sistema de recuperación de contraseña:** Implementar funcionalidad para recuperar contraseña mediante email.

Ampliaciones a medio plazo:

1. **Sistema de torneos:** Permitir crear torneos privados entre grupos de usuarios.
2. **Sistema de chat:** Implementar chat entre usuarios para compartir predicciones y estrategias.
3. **Gráficas avanzadas:** Añadir gráficas detalladas de rendimiento del usuario, evolución de puntos, etc.
4. **Sistema de logros:** Implementar badges y logros por diferentes hitos (primeras apuestas ganadas, rachas, etc.).
5. **Predicciones sobre estadísticas:** Permitir apostar sobre estadísticas individuales de jugadores (puntos, rebotes, asistencias).
6. **Modo multijugador en tiempo real:** Permitir competir en tiempo real durante partidos en curso.

Ampliaciones a largo plazo:

1. **Aplicación web:** Desarrollar versión web adicional usando React o Angular.
2. **Integración con redes sociales:** Permitir compartir logros y rankings en redes sociales.
3. **Sistema de ligas:** Crear ligas estacionales con premios virtuales.
4. **Análisis predictivo avanzado:** Implementar machine learning para mejorar las predicciones de cuotas.

-
- 5. **Sistema de streaming:** Integrar streaming de partidos en vivo (requiere licencias).
-

7. BIBLIOGRAFÍA Y RECURSOS ON-LINE

Bibliografía

- Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- JavaFX Documentation. <https://openjfx.io/>
- Android Developer Guide. <https://developer.android.com/guide>
- Jetpack Compose Documentation. <https://developer.android.com/jetpack/compose>
- MySQL Reference Manual. <https://dev.mysql.com/doc/>
- Material Design Guidelines. <https://material.io/design>

Recursos on-line

- Spring Boot Official Website: <https://spring.io/projects/spring-boot>
 - JavaFX Official Website: <https://openjfx.io/>
 - Android Developers: <https://developer.android.com/>
 - Gson User Guide: <https://github.com/google/gson/blob/main/UserGuide.md>
 - Jackson JSON Processor: <https://github.com/FasterXML/jackson>
 - MySQL Official Website: <https://www.mysql.com/>
 - Stack Overflow: <https://stackoverflow.com/>
 - GitHub: <https://github.com/>
-

8. GLOSARIO DE TÉRMINOS

8.1 Informáticos

- **API (Application Programming Interface):** Interfaz que permite la comunicación entre diferentes aplicaciones.
- **REST (Representational State Transfer):** Estilo arquitectónico habitual sobre HTTP; **no** es el mecanismo cliente↔backend de la versión entregada (ahí se usa TCP + JSON).
- **JSON (JavaScript Object Notation):** Formato de intercambio de datos ligero y legible.
- **ORM (Object-Relational Mapping):** Técnica de programación para convertir datos entre sistemas de tipos incompatibles.
- **JPA (Java Persistence API):** Especificación de Java para gestión de persistencia de datos.
- **DTO (Data Transfer Object):** Objeto que transporta datos entre procesos.
- **Repository:** Patrón de diseño que abstrae el acceso a datos.
- **Service:** Capa de lógica de negocio en arquitectura en capas.
- **Controller:** En JavaFX/Android, controladores de vistas. En el backend de este proyecto no hay **RestController** para la app; la capa de entrada remota es el servidor **socket**.
- **Compose:** Framework de Android para crear interfaces declarativas.

8.2 Problema

- **Apuesta virtual:** Predicción realizada con puntos ficticios.
- **Cuota:** Multiplicador que determina la ganancia potencial.

- **Puntos virtuales:** Moneda ficticia del sistema.
 - **Partido programado:** Partido que aún no se ha disputado.
 - **Partido finalizado:** Partido con resultado definitivo.
 - **Ranking:** Clasificación de usuarios por puntos.
 - **Win Rate:** Porcentaje de apuestas ganadas.
 - **Equipo local:** Equipo que juega en casa.
 - **Equipo visitante:** Equipo que juega fuera.
 - **Predicción:** Elección del usuario sobre el ganador (LOCAL o VISITANTE).
-

Fin del documento